Lezione 10 - Classi astratte e interfaccie

Introduzione: dalle gerarchie di classi alle astrazioni più evolute

Nelle lezioni precedenti sono stati introdotti i concetti fondamentali della programmazione orientata agli oggetti: **classi**, **oggetti**, **incapsulamento**, **ereditarietà** e **polimorfismo**.

Si è visto come l'ereditarietà consenta di costruire **gerarchie di classi**, favorendo il riuso del codice e la specializzazione del comportamento.
Il polimorfismo, come conseguenza dell'ereditarietà, **permette invece di trattare oggetti di classi diverse attraverso un riferimento comune, aumentando l'estendibilità del sistema senza modificare il codice esistente.**

A partire da queste basi, la lezione introduce strumenti più avanzati di astrazione: **classi astratte** e **interfacce**.

Entrambe consentono di definire comportamenti comuni che verranno concretizzati nelle sottoclassi, ma con finalità e livelli di vincolo differenti:

- le **classi astratte**:
  - **permettono di combinare metodi astratti e implementazioni concrete, oltre a poter definire attributi di istanza;**
- le **interfacce**:
  - **definiscono invece un contratto di comportamento puro, separando completamente la specifica dall'implementazione.**

Questi strumenti rappresentano un'evoluzione del concetto di ereditarietà, poiché permettono di progettare sistemi più flessibili, modulari e aderenti ai principi di buona progettazione, come la separazione delle responsabilità e l'estendibilità del software.

Classi astratte

Come in Python, le **classi astratte** sono utilizzate per modellare concetti molto **generici** all'interno di un'architettura software.
Essendo concetti **incompleti o astratti**, **non possono essere istanziate direttamente**:

- **non è possibile creare oggetti a partire da una classe astratta.**

Dichiarazione e regole principali

- **Una classe contenente uno o più metodi astratti deve essere dichiarata** `abstract`**.**
- **Anche una classe senza metodi astratti può essere dichiarata astratta, se si desidera impedirne l'istanziazione e usarla solo come classe base.**

> Esempio pratico: nell'esercizio della cartoleria, la classe `Articolo` era astratta perché rappresentava il concetto generico di prodotto.
> Le sottoclassi concrete `Penna` e `Gomma` fornivano invece implementazioni specifiche.

Metodi astratti

Un **metodo astratto** è un metodo **privo di implementazione:**

- **definisce solo la firma.**
- **L'implementazione concreta viene demandata alle sottoclassi.**

Per poter creare oggetti, è necessario definire una **sottoclasse concreta** che:

1. **Estenda la classe astratta.**
2. **Implementi tutti i metodi astratti.**

Solo a partire da questa sottoclasse sarà possibile istanziare oggetti.

Variabili di tipo astratto

Anche se non è possibile creare oggetti della classe astratta, **si possono dichiarare variabili** di quel tipo, sfruttando così il polimorfismo.
In questo modo, una reference di tipo astratto può puntare a qualsiasi oggetto di sottoclasse concreta.

Struttura di una classe astratta

Una classe astratta può contenere:

- **Attributi**
- **Metodi concreti**
- **Metodi astratti**

- **Costruttori** (utilizzati per inizializzare lo stato comune alle sottoclassi)

Esempio: classe `Message`

Java

```java
public abstract class Message {
    private String sender;

    public Message(String from) {
        sender = from;
    }

    public abstract void send(); // metodo astratto

    public String getSender() {
        return sender;
    }
}
```

Analisi dettagliata:

1. **Dichiarazione della classe**

Java

```java
public abstract class Message {
```

- La classe `Message` è dichiarata `abstract` perché rappresenta un concetto **generico** di messaggio.
- Non descrive un messaggio concreto (email, SMS, notifica, ecc.), ma solo le caratteristiche comuni a tutti i messaggi.
- Di conseguenza, **non è possibile creare oggetti direttamente di tipo** `Message`.

2. **Attributo della classe**

Java

```java
private String sender;
```

- L'attributo `sender` rappresenta il **mittente del messaggio**.

- È dichiarato `private` per rispettare il principio di **incapsulamento**, impedendo l'accesso diretto dall'esterno della classe.
- Il valore potrà essere letto solo tramite un metodo pubblico.

3. **Costruttore**

Java

```java
public Message(String from) {
    sender = from;
}
```

- Anche se la classe è astratta, **può e deve avere un costruttore**.
- **Questo costruttore serve a inizializzare lo stato comune a tutte le sottoclassi, in questo caso il mittente del messaggio.**
- **Il costruttore verrà chiamato implicitamente quando si istanzierà una sottoclasse concreta di `Message`**.

4. **Metodo astratto**

Java

```java
public abstract void send();
```

- Il metodo `send()` è dichiarato `abstract` perché la classe `Message` **non conosce il modo concreto in cui il messaggio verrà inviato**.
- L'invio può variare a seconda del tipo di messaggio (email, SMS, push notification, ecc.).
- Questo implica che:
  - il metodo **non ha un corpo**
  - ogni sottoclasse concreta **è obbligata a fornire un'implementazione** di `send()`

Significato complessivo dell'esempio

La classe `Message` definisce:

- **uno stato comune** (il mittente)
- **un comportamento comune** (il metodo `getSender`)
- **un comportamento astratto** (`send`) che deve essere specializzato

In questo modo, `Message` funge da **classe base astratta**, utile per modellare una gerarchia di classi di messaggi diversi, mantenendo una struttura coerente e favorendo il **polimorfismo**.

Le interfacce

Oltre alle classi astratte, in Java esistono le **interfacce:**

- **permettono di definire un contratto formale tra più entità anche non collegate da una relazione di ereditarietà.**

Un'interfaccia definisce cosa deve essere fatto, senza specificare come farlo.
Un'interfaccia consente a classi diverse, anche appartenenti a gerarchie differenti, di **interagire attraverso un insieme comune di metodi**, senza conoscere i dettagli della loro implementazione.
In questo senso, l'interfaccia funge da **collegamento** tra:

- **una o più classi che utilizzano i metodi definiti dall'interfaccia;**
- **una o più classi che implementano tali metodi.**

In pratica, l'interfaccia stabilisce *cosa* deve essere fatto, mentre le classi che la implementano decidono *come* farlo.
Questo favorisce un forte **disaccoppiamento** tra le parti del sistema e rende il codice più flessibile e manutenibile.

Di solito, si ha:

- **una classe (o più classi) che lavora in termini di interfaccia, cioè utilizza i metodi dichiarati nell'interfaccia;**
- una o più classi concrete che **implementano l'interfaccia**, fornendo l'implementazione effettiva dei metodi.

Analogie

Un'analogia efficace è quella delle **certificazioni professionali**:
la certificazione rappresenta un insieme di competenze standardizzate. Il datore di lavoro non ha bisogno di sapere *come* il lavoratore ha acquisito quelle competenze, ma solo che possiede la certificazione.
Allo stesso modo, una classe che utilizza un'interfaccia non ha bisogno di

conoscere la classe concreta che la implementa, ma solo che rispetti il contratto definito dall'interfaccia.

Un'altra metafora è quella delle **due sponde di un fiume**:
le sponde rappresentano entità indipendenti, mentre l'interfaccia è il **ponte** che permette la comunicazione tra di esse senza fonderle in un'unica struttura.

Caratteristiche principali delle interfacce in Java

In Java, un'interfaccia:

1. Rappresentano una **categoria completamente astratta**.
2. **Dichiarano solo metodi astratti** (nelle versioni più recenti del linguaggio possono esistere anche metodi `default` e `static`, ma il principio rimane quello del contratto).
3. Non possono essere istanziate.
4. Vengono **implementate** da una o più classi tramite la parola chiave `implements`.
5. **Non possiedono attributi di istanza**; possono contenere solo **costanti** `public static final`.
6. **Non hanno costruttori**, nemmeno quello di default, perché non è possibile creare oggetti direttamente da un'interfaccia.

Esempio di caso d'uso: quando scegliere tra classe astratta e interfaccia

Supponiamo di dover modellare una classe generica in Java e di dover scegliere se utilizzare una **classe astratta** oppure un'**interfaccia**.
La scelta dipende principalmente dalle caratteristiche che vogliamo definire.

- **Classe astratta**:
  - Condivide **attributi comuni** tra le sottoclassi.
  - Fornisce **metodi concreti** con implementazione.
  - È adatta quando esiste uno **stato condiviso** e una base di codice comune.
- **Interfaccia**:
  - Definisce **solo comportamenti** (metodi).

    - Non ha stato condiviso né implementazioni concrete (fatta eccezione per i metodi `default` o `static`).
    - È adatta quando si vuole imporre un **contratto uniforme** a classi anche molto diverse tra loro, favorendo il **disaccoppiamento** e il **polimorfismo**.

In sintesi:

- La **classe astratta** serve a modellare:
    - **entità generiche con attributi e comportamenti condivisi, che possono essere specializzati nelle sottoclassi.**
- L'**interfaccia** serve a definire:
    - **un insieme di comportamenti comuni da implementare in classi anche completamente indipendenti tra loro, senza condividere stato.**

> In altre parole, l'interfaccia è uno strumento avanzato di progettazione che si appoggia su concetti di base di **ereditarietà** e **polimorfismo**, permettendo di superare i limiti della **mancata ereditarietà multipla** in Java.

Perché usare un'interfaccia se posso usare una classe astratta?

- È vero che sia **classe astratta** che **interfaccia** possono definire **solo metodi astratti**, cioè senza implementazione.
- La **differenza principale** sta nello **stato e nei costruttori**:
    - **Una classe astratta può avere attributi di istanza e un costruttore. Questo permette di definire uno stato condiviso e inizializzazioni comuni per tutte le sottoclassi.**
    - **Un'interfaccia non ha costruttori né attributi di istanza; può contenere solo costanti (`public static final`) e serve unicamente a definire un contratto di metodi che le classi concrete dovranno implementare.**

In altre parole:

- **Classe astratta:** stato + comportamento + possibilità di avere implementazioni comuni.
- **Interfaccia:** solo comportamento, nessuno stato, serve a garantire polimorfismo e disaccoppiamento tra classi non correlate.

> Quindi la scelta dipende dal **bisogno di condividere stato** o meno.
> **Se vuoi solo imporre un contratto di metodi, usa un'interfaccia;**
> **se invece vuoi fornire codice e stato comune, usa una classe astratta**.

Sintassi delle interfacce in Java

Java 7

In Java 7 un'interfaccia può dichiarare solo:

- **Metodi pubblici e astratti** (`public abstract`)
- **Costanti di classe** (`public static final`)

Per dichiarare un'interfaccia si utilizza la parola chiave `interface` al posto di `class`.

Esempio:

Java

```java
public interface OggettoPrezzabile {
    void setPrezzo(double prezzo);  // astratto e pubblico sottointeso
    double getPrezzo();
}
```

Note importanti:

- In un'interfaccia, `abstract` **è sottinteso** per tutti i metodi, quindi scriverlo esplicitamente non è necessario.
- `public` è obbligatorio per i metodi dell'interfaccia.
- Le costanti dichiarate sono `public static final`:

Java

```java
public static final double IVA = 0.22;
```

Java 8

A partire da Java 8, le interfacce hanno introdotto alcune novità:

- Possono avere **metodi concreti** usando la keyword `default`.
- Possono avere **metodi statici**, accessibili tramite il nome dell'interfaccia.

Esempio:

```java
public interface OggettoPrezzabile {
    double getPrezzo();

    // metodo concreto con implementazione di default
    default void stampaPrezzo() {
        System.out.println("Prezzo: " + getPrezzo());
    }

    // metodo statico
    static void saluta() {
        System.out.println("Benvenuto nell'interfaccia OggettoPrezzabile");
    }
}
```

Sintesi:

- I metodi senza implementazione restano **astratti**.
- I metodi `default` permettono di avere un comportamento condiviso senza forzare le sottoclassi a implementarlo.
- I metodi `static` sono legati all'interfaccia stessa, non agli oggetti che la implementano.

Regole e utilizzo delle interfacce

Le **interfacce** rappresentano uno strumento avanzato di **progettazione e sviluppo software**, basato sui principi fondamentali della **programmazione a oggetti**, in particolare **ereditarietà** e **polimorfismo**.

Esse consentono di superare alcune limitazioni del linguaggio, in particolare:

- **Ereditarietà multipla**: **una classe può implementare più interfacce, aggirando il vincolo che limita l'ereditarietà a una sola superclasse in Java.**
- **Definizione di comportamenti astratti**: **un'interfaccia descrive *cosa* deve essere fatto, lasciando alle classi concrete la responsabilità di definire *come* realizzarlo.**

Principi fondamentali

1. **Implementazione obbligatoria dei metodi**
   - **Una classe che implementa un'interfaccia deve fornire l'implementazione di tutti i metodi dichiarati.**
   - Se non lo fa, la classe diventa automaticamente **astratta**.
2. **Nessuna istanziazione diretta**
   - **Le interfacce non hanno costruttori e non possono essere istanziate con `new`.**
   - Tuttavia, è sempre possibile dichiarare variabili del tipo dell'interfaccia, analogamente a quanto avviene con le classi astratte.
     - In questo caso, la variabile conterrà un riferimento a un oggetto che implementa l'interfaccia.
3. **Polimorfismo a runtime**
   - **Utilizzando variabili di tipo interfaccia, i metodi possono essere invocati senza conoscere la classe concreta dell'oggetto.**
   - **A runtime, la JVM determinerà l'implementazione specifica da eseguire in base al tipo reale dell'oggetto, garantendo flessibilità e riusabilità del codice.**

Analogia: Parentela Debole

Le interfacce possono essere pensate come una **parentela debole** tra oggetti:

- Un oggetto non ha un legame diretto con una vera superclasse (come un oggetto con il tipo `Object`),
- Ma condivide **comportamenti comuni** con altri oggetti che implementano la stessa interfaccia.

Esempio:

- Se io e i miei colleghi frequentiamo un corso di Java, possiamo dire di avere una "parentela debole" come programmatori Java.
- Non siamo parte di una stessa famiglia (classe concreta), ma apparteniamo a una categoria comune definita dall'interfaccia.

Ereditarietà multipla tramite interfacce

- Una classe può **ereditare da una sola superclasse** (o da `Object` se non specificata) e **implementare più interfacce**.
- Sintassi:

```java
public class Libro extends Media implements OggettoPrezzabile, OggettoSfogliabile {
    ...
}
```

Interfacce in UML

In UML, la relazione tra **classe** e **interfaccia** è rappresentata con:

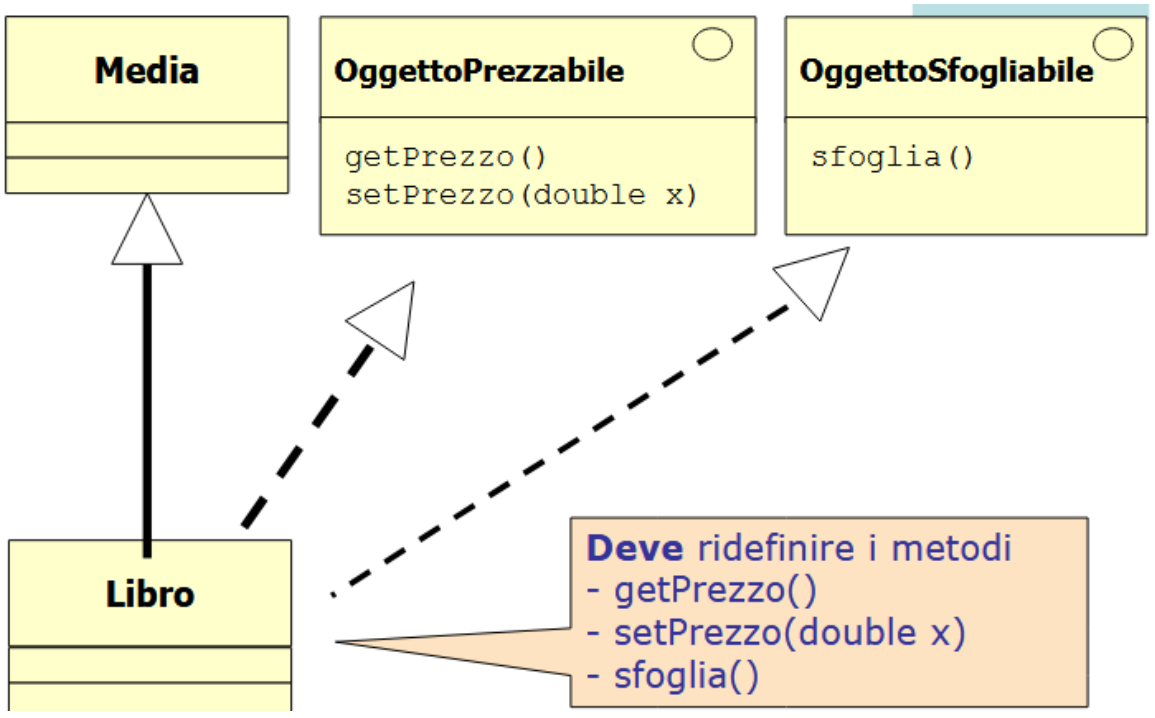
- **Linea tratteggiata**
- **Freccia vuota rivolta verso l'interfaccia**

Questa simbologia indica che la classe **implementa** l'interfaccia, creando un legame di tipo **is-a debole:**

- **cioè un contratto di comportamento senza ereditarietà concreta di stato o costruttori.**

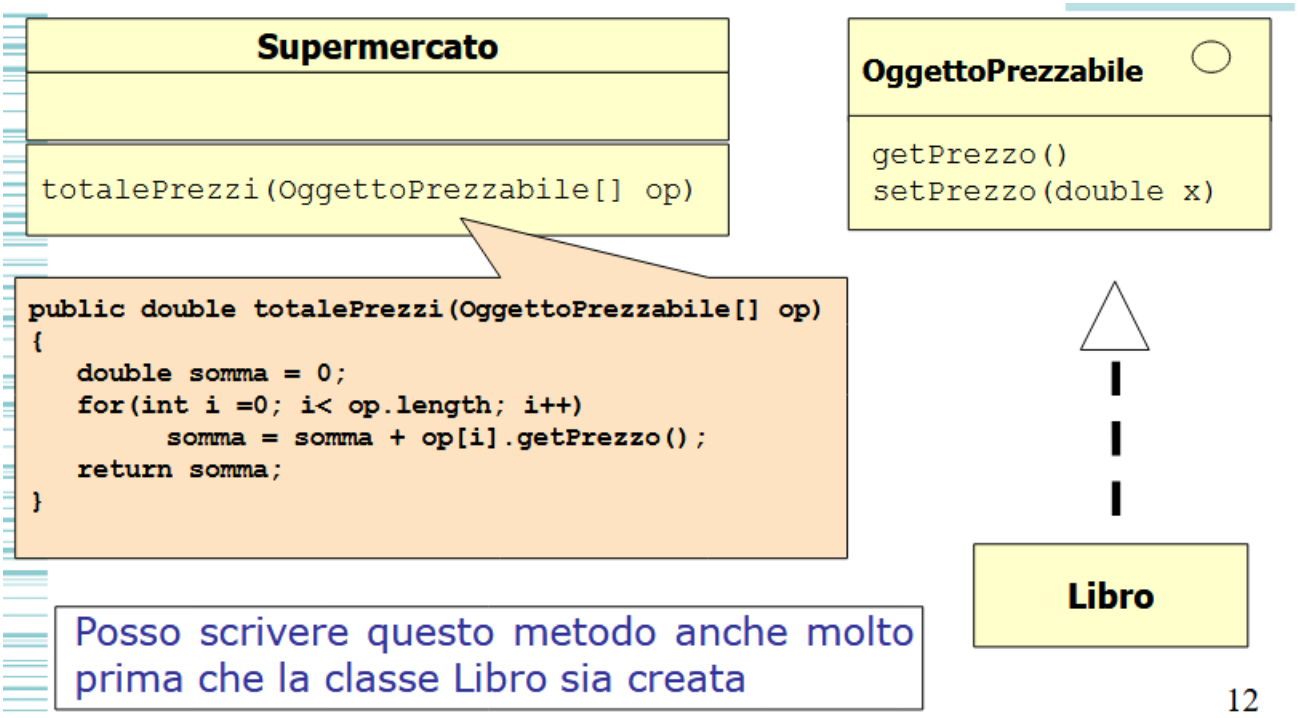
Esempio pratico: `Libro`

Consideriamo il diagramma seguente:



- `Libro` **estende** la classe `Media` → **relazione is-a forte, eredita stato e comportamenti concreti.**
- `Libro` **implementa** le interfacce `OggettoPrezzabile` e `OggettoSfogliabile` → **relazioni is-a debole, obbligando la classe a rispettare i contratti definiti dalle interfacce.**

Sintesi grafica con UML



Interpretazione:

- **Le linee continue con freccia piena rappresentano ereditarietà forte (classe concreta → superclasse).**
- **Le linee tratteggiate con freccia vuota rappresentano implementazione di interfacce (is-a debole)**.
- In questo modo, `Libro` combina **stato e comportamenti ereditati da Media** con **comportamenti astratti definiti dalle interfacce**, ottenendo sia polimorfismo che flessibilità di progettazione.

Interfacce funzionali

Un'interfaccia funzionale **è un'interfaccia che contiene un solo metodo astratto.**
Questo singolo metodo rappresenta **il comportamento principale** che quell'interfaccia vuole modellare.

- **Può avere anche metodi default o statici, ma solo un metodo astratto.**

- **Il metodo astratto è quello che viene implementato tramite lambda expression.**
  Sintassi generica

```java
@FunctionalInterface
public interface NomeInterfaccia {
    void metodoUnico(); // metodo astratto
}
```

> **L'annotazione `@FunctionalInterface` non è obbligatoria, ma serve a indicare chiaramente che l'interfaccia è pensata per essere funzionale.**
>
> Il compilatore segnalerà un errore se ci sono più di un metodo astratto.

Estensioni di interfacce

In Java è possibile **creare interfacce che specializzano altre interfacce**, in modo simile a come le classi astratte possono essere estese dalle sottoclassi. Questo permette di costruire gerarchie di interfacce e definire comportamenti sempre più specifici senza dover scrivere codice concreto.
Esempio di base

```java
public interface Moveable {
    public void move(double x, double y);
}

public interface Powered extends Moveable {
    public String powerSource();
}
```

Spiegazione:

- `Moveable` **definisce un comportamento generico: la possibilità di muoversi nello spazio.**
- `Powered` estende `Moveable`, **aggiungendo un nuovo metodo** `powerSource()`**, cioè una specializzazione dell'interfaccia precedente.**

- Una classe concreta che implementa `Powered` **dovrà fornire l'implementazione di entrambi i metodi, sia** `move()` **sia** `powerSource()`**.**

Caratteristiche principali

1. **Ereditarietà multipla per le interfacce**
   - A differenza delle classi, **un'interfaccia può estendere più interfacce contemporaneamente, combinando più comportamenti astratti in un'unica interfaccia.**
   - Questo risolve il problema della mancata ereditarietà multipla tra classi in Java.
2. **Obbligo di implementazione**
   - **Qualsiasi classe che implementa un'interfaccia derivata deve fornire l'implementazione di tutti i metodi dichiarati nelle super-interfacce.**
     Esempio con ereditarietà multipla tra interfacce

Java

```java
public interface AdvancedRobot extends Powered, Repairable {
    public void selfCheck();
}
```

- `AdvancedRobot` **eredita tutti i metodi di** `Powered` **e di** `Repairable`**.**
- Una classe concreta che implementa `AdvancedRobot` **dovrà implementare tutti i metodi ereditati dalle interfacce** `Powered` **e** `Repairable`**, oltre a** `selfCheck()`**.**

> In questo modo, le interfacce permettono di creare un **contratto flessibile e modulare**, che può essere esteso e combinato senza vincolare le classi concrete a una singola gerarchia.

Marker Interface

Le **marker interface** sono un tipo speciale di interfaccia che **non dichiara metodi**.
Il loro scopo NON è quello di fornire comportamenti da implementare, ma **fornire informazioni sulla natura della classe** che le implementa. In altre parole, servono solo come **etichetta** o **marcatore**.

Caratteristiche principali

- **Non richiedono l'implementazione di metodi**; **la classe che le implementa non deve fare nulla di speciale.**
- **Forniscono informazioni al sistema o alla JVM su come trattare gli oggetti.**
- **Hanno scopo puramente identificativo**: **la loro presenza segnala che la classe possiede una certa proprietà o capacità.**

Esempio: `Cloneable`

Java

```java
public class Documento implements Cloneable {
    private String testo;

    // metodi della classe
}
```

- Implementando `Cloneable`, **la classe** `Documento` **dichiara di essere clonabile, cioè che gli oggetti istanziati da essa possono essere duplicati.**
- Tuttavia, la **marker interface non obbliga a implementare il metodo** `clone()`; spetta alla classe fornire un'implementazione se necessario.

> **Le marker interface sono utili per segnalare capacità o caratteristiche particolari senza definire metodi concreti, facilitando la progettazione flessibile e il polimorfismo basato su etichette.**

Clonazione di oggetti

In Java, la **clonazione** di un oggetto consente di creare una copia indipendente dell'oggetto stesso.
Questo è particolarmente utile per evitare di restituire riferimenti diretti a campi privati, preservando così l'incapsulamento.

Come funziona

- Ogni classe può definire la propria modalità di clonazione **implementando il metodo** `clone()`.
- Il metodo `clone()` da implementare **override** quello protetto (`protected`) definito nella classe `Object`:

```java
@Override
public Object clone() {
    // logica di clonazione
}
```

Considerazioni pratiche

- Per permettere la clonazione, la classe **deve implementare l'interfaccia** `Cloneable`; altrimenti, la JVM solleverà una `CloneNotSupportedException`.
- In genere, si implementa `clone()` per restituire una copia **profonda o superficiale** dell'oggetto, a seconda delle necessità:
  - **Shallow copy**: **copia dei riferimenti agli oggetti interni;**
  - **Deep copy**: **copia completa degli oggetti interni, creando oggetti indipendenti.**

> La clonazione è uno strumento potente per **gestire oggetti in sicurezza**, evitando modifiche indesiderate da altre parti del programma.